

操作系统课程设计报告

基于虚拟磁盘的简易二级文件系统设计与实现

课程名称	操作系统课程设计
实验题目	实验二 Linux 二级文件系统
项目名称	mini-kv OSFS course filesystem
学生姓名	【请填写】
学号	【请填写】
班级	【请填写】
指导教师	【请填写】
完成日期	2026 年 7 月

摘要

本课程设计围绕“Linux 二级文件系统”实验展开，目标是在不修改操作系统内核的前提下，用一个二进制文件模拟磁盘空间，并在其上实现一个简化但结构清楚的多用户文件系统。系统被命名为 OSFS，作为 mini-kv 项目的独立旁路模块存在。它不接入原来的 KV、WAL、TCP 服务链路，而是通过 minikv_osfs 命令程序提供 format、login、dir、create、delete、open、read、write、close、chmod 等演示入口。这样处理的好处是边界清楚：课程设计要解释文件系统，原项目核心仍然保持稳定。

OSFS 的磁盘布局包含超级块、块位图、inode 表、根目录和数据块。文件名保存在目录项中，文件类型、权限、owner、长度、时间戳和直接数据块号保存在 inode 中，文件正文则写入数据块。权限方面，系统实现了 root 用户、普通用户、owner 位和 other 位的基本读写控制。测试部分覆盖格式化、持久化、目录列出、文件读写、权限拒绝、空间不足失败保持旧内容等行为，能够对应课设要求中的“二进制文件模拟磁盘”“inode 信息”“超级块信息”“文件信息”“用户登录”和“读写保护”。

关键词：操作系统；二级文件系统；虚拟磁盘；inode；权限控制；C++

1 选题来源与需求分析

课程设计要求中，实验二要求模仿 Linux EXT2 文件系统，用二进制文件模拟磁盘空间，保存用户信息、inode 信息、超级块信息和文件信息，并实现 dir、create、delete、open、close、read、write、login 等命令。这个题目比单纯写一个命令行工具更接近操作系统课程的核心，因为它迫使程序把“文件”拆成元数据和数据块两部分来处理。学习这部分之前，文件往往只是一个路径和一段内容；做完这个设计后，会更容易理解为什么真实文件系统需要位图、inode 表和目录项这些结构。

mini-kv 原本已经有 WAL、snapshot、restore 等存储相关能力，但它的主任务是键值存储，不适合直接伪装成操作系统文件系统。如果为了课设把目录、权限、inode 全塞进 KV 命令里，项目会变得混乱。因此本设计选择增加一个独立 OSFS 层：它复用仓库已有的 CMake、CTest 和工程纪律，但运行入口、磁盘镜像和测试都与 KV 主线隔离。这样既能满足课设要求，也不会把原项目变成一个难维护的混合系统。

要求或模块	本项目中的对应实现
二进制文件模拟文件系统	VirtualDisk 按固定块大小创建和读写 osfs-course.img。
超级块信息	Block 0 保存 magic、block size、block count、inode count、inode 表起止位置和 root inode。
block/inode 管理	Block bitmap 记录数据块占用；inode 表保存每个文件或目录的元数据。

文件和目录操作	CommandProcessor 暴露 DIR、CREATE、DELETE、OPEN、READ、WRITE、CLOSE。
用户交互	minikv_osfs 提供交互 shell；LOGIN 切换 root 或普通用户。
读写保护	CHMOD 修改保护码；READ/WRITE 根据 owner/other 权限判断是否允许。
系统测试	osfs_tests、osfs_cli_smoke 和全量 CTest 共同验证功能与边界。

2 总体结构设计

OSFS 分成三层：最外层是命令处理层，负责把用户输入的字符串转成具体文件操作；中间层是 FileSystem，负责维护超级块、位图、inode、目录项和权限；最底层是 VirtualDisk，它只关心按块读写二进制文件。这种分层让报告和代码都比较容易解释。比如 WRITE 命令失败时，不需要在命令层猜测磁盘状态，而是由 FileSystem 统一判断权限和空间；VirtualDisk 也不需要理解文件名，只处理块号和字节数组。

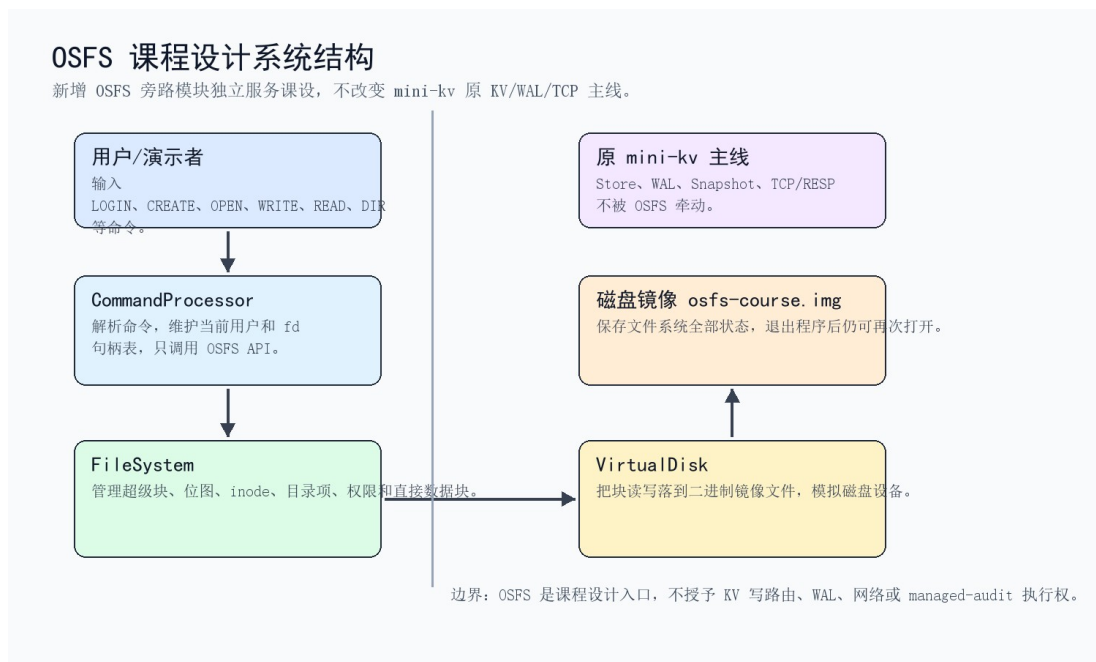


图 1 OSFS 课程设计系统结构

3 虚拟磁盘与数据结构设计

虚拟磁盘采用固定块组织。当前默认块大小为 512 字节，演示时通常创建 96 或 256 个块。Block 0 是超级块，Block 1 是块位图，后面一段区域保存 inode 表，剩余区域作为数据块。根目录本身也是一个目录 inode，它的数据块里保存目录项。目录项只保存文件名、inode 编号和是否有

效；真正的文件长度、权限、owner、时间戳和数据块列表都在 inode 中。这样的设计比直接把“文件名和内容”连在一起更接近课程中讲到的文件系统思路。

为了让实现可控，本版只使用直接数据块。真实 EXT2 inode 会包含 12 个直接块以及多级间接块，但课设演示重点是理解文件系统内部结构，而不是追求大文件容量。直接块方案更适合课堂答辩：它能清楚展示位图如何分配块、inode 如何记录块号、目录如何通过 inode 找到文件，同时避免间接块把讲解重点拉得过散。

虚拟磁盘布局

二进制文件被切成固定大小块，OSFS 在这些块上组织元数据和文件内容。

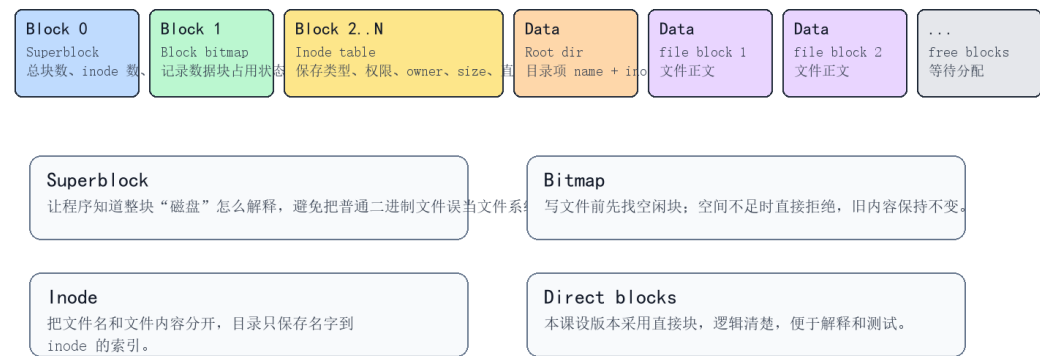


图 2 OSFS 虚拟磁盘布局

4 核心操作流程

格式化时，程序先创建指定大小的二进制文件，再写入超级块、初始化位图、清空 inode 表，并创建 root 目录。创建文件时，系统分配一个空闲 inode，在根目录中添加目录项。打开文件时，命令层记录一个 fd 到 inode 的映射；后续 READ 和 WRITE 通过 fd 找回 inode。删除文件时，系统释放 inode 和它占用的数据块，并把目录项标记为无效。

WRITE 是最值得说明的流程。它不只是把字符串写到文件末尾，而是先检查 fd 是否存在、打开模式是否允许写、当前用户是否有写权限，然后计算新内容需要多少数据块。空间不足时，程序会在释放旧块之前直接返回错误，这一点在测试中专门覆盖。这个细节看起来不大，但它说明实现时考虑了失败场景，不会出现写一半失败导致旧文件被破坏的问题。

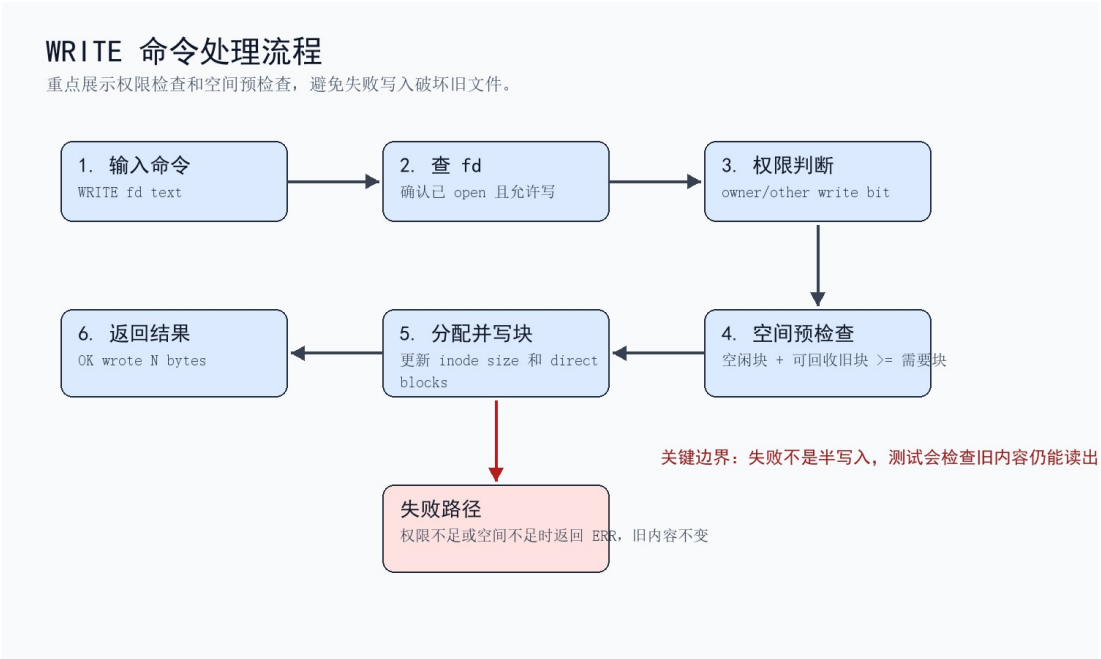


图 3 WRITE 命令处理流程

5 权限与用户模型

用户模型故意保持简单。root 用户 uid 为 0，普通用户名通过稳定哈希得到 uid。每个文件保存 owner 和八进制保护码，例如 0644 或 0600。读写判断时，root 直接通过；如果当前用户是 owner，则检查 owner read/write 位；如果不是 owner，则检查 other read/write 位。本设计没有引入 group，是因为课程要求强调读写保护，owner/other 已经足以演示保护码的作用。

演示中可以先用 root 创建并写入 report 文件，再把权限改成 0600，随后 LOGIN guest 并尝试 READ。guest 不是 owner，other read 位又被关闭，因此系统返回 permission denied。这条演示路径比单纯展示 chmod 更有说服力，因为它把用户切换、保护码和读操作连接在一起，能说明权限不是文档里的字段，而是实际参与了命令执行。

6 关键模块说明

要求或模块	本项目中的对应实现
include/minikv/osfs/virtual_disk.hpp	声明虚拟磁盘接口，提供 create、open、read_block、write_block。
src/osfs_virtual_disk.cpp	负责磁盘镜像大小校验、块边界校验和二进制读写。
include/minikv/osfs/filesystem.hpp	声明文件系统 API、FileInfo、DirectoryEntry、Stat、OpenMode 等类型。
src/osfs_filesystem.cpp	实现超级块、位图、inode 表、目录项、文件读写、权限和空间分配。

include/minikv/osfs/command_processor.hpp	声明课程演示命令处理器和命令返回结构。
src/osfs_command_processor.cpp	解析 LOGIN、DIR、CREATE、OPEN、WRITE、READ、CHMOD 等命令并维护 fd 表。
src/osfs_main.cpp	提供 minikv_osfs 命令行入口，支持 --disk、--format、--blocks、--script。
tests/osfs_tests.cpp	用断言覆盖持久化、权限、目录行、空间失败等关键行为。

7 测试与演示结果

本设计不只依赖一次手工演示。自动化测试先验证底层行为，例如格式化后重新打开磁盘仍能读到文件，chmod 后普通用户会被拒绝，空间不足时旧内容不被破坏。脚本 smoke 再从 CLI 入口跑一遍接近课堂演示的命令序列。最后，全量 CTest 确认新增 OSFS 没有破坏 mini-kv 原有 KV、WAL、snapshot、TCP 和证据链测试。

```
03 OSFS course demo transcript
mini-kv v1630 coursework archive evidence

mini-kv osfs course shell
disk=D:\mini-kv\make-build-v1630\osfs-course-demo.img blocks=96 block_size=512 inodes=64
Commands: HELP, WHOAMI, LOGIN user, DIR, CREATE name, DELETE name, STAT name, CHMOD name mode, OPEN name r|w|rw, READ fd, WRITE fd text, CLOSE fd,
QUIT
osfs> WHOAMI
root uid=0
osfs> DIR
(empty)
osfs> CREATE report
OK created report
osfs> OPEN report rw
OK fd=3
osfs> WRITE 3 course design storage layer
OK wrote 27 bytes
osfs> READ 3
course design storage layer
osfs> CLOSE 3
OK closed 3
osfs> CHMOD report 0600
OK chmod report 0600
osfs> LOGIN guest
OK login guest uid=3193756059
osfs> OPEN report r
ERR permission denied
osfs> DIR
name inode physical mode owner size
report 1 15 0600 0 27
osfs> LOGIN root
OK login root uid=0
osfs> STAT report
name=report inode=1 physical=15 mode=0600 owner=0 size=27 created_at=1783306760 modified_at=1783306760
osfs> QUIT
BYE
```

图 4 OSFS 命令行演示结果

演示输出中可以看到 CREATE、OPEN、WRITE、READ、CHMOD、LOGIN、权限拒绝和 DIR 列表都按预期执行。其中 report 文件在 chmod 0600 后，guest 用户读取失败，说明保护码不是静态展示字段。

```
05 Full CTest summary
mini-kv v1630 coursework archive evidence

Command: ctest --test-dir cmake-build-v1630 --output-on-failure

Result:
- Full local CTest completed successfully.
- 344/344 tests passed
- 0 tests failed
- Total Test time (real) = 150.80 sec.

Relevant early tests:
- osfs_tests passed
- osfs_cli_smoke passed.

Relevant tail tests:
- no_network_safety_fixture_contract_receipt_tests passed.
- abort_rollback_semantics_contract_receipt_tests passed.
- input_hardening_candidate_gate_receipt_tests passed

Meaning:
v1630's coursework deliverables do not change runtime behavior, and the local full test suite confirms the OSFS delivery package did not disturb the
existing mini-kv command, WAL, snapshot, TCP/RESP, receipt, and archive guard surfaces.
```

图 5 全量 CTest 验证证据

8 运行与复现步骤

在 Linux 环境中，可以使用 CMake 构建 minikv_osfs 目标，然后用脚本运行演示命令。Windows 本地验证使用 CLion 捆绑 MinGW 工具链完成，GitHub Actions 也覆盖 Ubuntu、macOS 和 Windows 构建测试。课堂演示时建议只展示 OSFS 入口，不需要启动 mini-kv TCP 服务。

```
cmake -S . -B build
cmake --build build --target minikv_osfs
./build/minikv_osfs --disk data/osfs-course.img --format --blocks 96 --script
tests/osfs_smoke_script.txt
```

9 不足与改进方向

当前 OSFS 适合作为课程设计主体，但它不是完整生产文件系统。它只实现根目录，没有多级目录；inode 使用直接数据块，没有间接块；权限模型也只区分 owner 和 other，没有 group。笔者认为这些限制是可以接受的，因为本次课程设计的关键是讲清楚文件系统内部结构和命令行为，而不是复刻完整 EXT2。

如果继续扩展，比较合理的顺序是先加 mkdir、cd、pwd，把根目录扩成多级目录；随后再加 group 权限和间接块。另一个方向是增加 fsck 类检查命令，用来发现位图、inode 表和目录项之间的不一致。这样的扩展会更接近真实文件系统维护，但也会显著增加测试和答辩解释成本。

10 总结

通过这次设计，我对文件系统的理解从“文件路径加文件内容”推进到了“磁盘块、元数据和权限共同工作”。OSFS 的实现不复杂，但它把课程中的几个关键点连在了一起：超级块说明磁盘如何被解释，位图说明空间如何被分配，inode 说明文件属性和数据块如何绑定，目录项说明文件名如何映射到 inode，权限检查则说明多用户环境下为什么不能只关心数据是否存在。

这版设计也保留了工程上的克制。它没有把 OSFS 混进 mini-kv 原来的 KV 命令，也没有启动网络服务或打开额外执行链路。对课程设计来说，这使系统边界更好讲；对原项目来说，也避免了为了完成作业而破坏已有结构。

参考资料

操作系统课程设计要求 2026，课程讲义 PDF。

Linux Kernel 源码目录与文件系统子系统课程资料。

mini-kv 项目源码：`include/minikv/osfs`、`src/osfs_*.cpp`、`tests/osfs_tests.cpp`。